

AWS S3 Data Processing: S3 CSV File Chunker Pipeline

1. Overview

This guide documents the architecture, configuration, and implementation for splitting large CSV datasets stored in Amazon S3 into smaller files (chunks of 5,000 rows each) and uploading them back to S3.

It also provides instructions for automating this workflow using **GitHub Actions**.

2. Prerequisites & Dependencies

Local / CLI Environment Setup

- **AWS CLI (v2)**
- **Python 3.8+**

Required Python Libraries

To stream files to and from AWS S3, install pandas and s3fs:

```
pip install pandas s3fs
```

- **pandas**: Handles reading data in chunks using chunksize.
- **s3fs**: Interface built on boto3 that allows Pandas to interact directly with S3 URIs (s3://...).

3. AWS Configuration

Step 3.1: Create S3 Bucket and Folders

1. Open the **Amazon S3 Console**.
2. Create an S3 bucket (e.g., big-file-chunks) in your target region (e.g., eu-north-1).
3. Create two folder prefixes inside the bucket:
 - input_files/
 - output_files/
4. Upload your source CSV file to s3://big-file-chunks/input_files/.

Step 3.2: Create IAM User & Permissions

1. Open the **IAM Console** > **Users** > **Create user**.
2. Set the username (e.g., macbook-s3-user) and leave AWS Management Console access **unchecked**.

3. Attach an inline or custom policy granting minimal read/write access to this specific bucket:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3ListBucketPermissions",
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::big-file-chunks"
    },
    {
      "Sid": "S3ReadWriteObjectPermissions",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::big-file-chunks/*"
    }
  ]
}
```

Step 3.3: Generate Access Credentials

1. Under the newly created user, navigate to the **Security credentials** tab.
2. Under **Access keys**, click **Create access key**.
3. Choose **Command Line Interface (CLI)** and copy/download the **Access Key ID** and **Secret Access Key**.

Step 3.4: Configure Credentials Locally

In your terminal, run:

```
aws configure
```

Provide the required information when prompted:

```
AWS Access Key ID [None]: <YOUR_ACCESS_KEY_ID>
AWS Secret Access Key [None]: <YOUR_SECRET_ACCESS_KEY>
Default region name [None]: eu-north-1
Default output format [None]: json
```

Note: The region must match the region code where your bucket is hosted (e.g., eu-north-1).

4. Python Script Implementation

Save the following code as `file_chunker_python.py`:

```
import pandas as pd

# Define S3 input and output locations
input_path =
"s3://big-file-chunks/input_files/annual-enterprise-survey-2025-financial-year-provisional-size-ba
nds.csv"
output_dir = "s3://big-file-chunks/output_files"

print(f"Reading from: {input_path}")

# Stream input directly from S3 in 5000-row chunks
for i, chunk in enumerate(pd.read_csv(input_path, chunksize=5000, dtype=str)):
    target_file = f"{output_dir}/chunk_{i + 1}.csv"
    chunk.to_csv(target_file, index=False)
    print(f"Uploaded: {target_file}")

print("Done!")
```

How It Works

1. **Memory Safety:** Setting `chunksize=5000` tells Pandas to return an iterator rather than reading the entire file into memory.
2. **Type Consistency:** `dtype=str` ensures values are not incorrectly inferred as floats or integers across different chunks.
3. **S3 Streaming:** `s3fs` streams data directly between RAM and S3 without writing intermediate files to disk.

5. Automated Execution via GitHub Actions

You can automate this script to run on a schedule, manually via workflow dispatch, or whenever code is pushed to your repository using GitHub Actions.

Step 5.1: Configure Secrets in GitHub

Never commit your AWS credentials to a Git repository. Store them as repository secrets instead:

1. Open your GitHub repository.
2. Navigate to **Settings > Secrets and variables > Actions**.
3. Click **New repository secret** and add the following three secrets:

Secret Name	Value
AWS_ACCESS_KEY_ID	Your IAM user Access Key ID
AWS_SECRET_ACCESS_KEY	Your IAM user Secret Access Key
AWS_REGION	The region code (e.g., eu-north-1)

Step 5.2: Create the GitHub Actions Workflow

Inside your GitHub repository, create a directory structure `.github/workflows/` and add a file named `s3_chunker.yml`.

name: Run S3 File Chunker Pipeline

on:

 workflow_dispatch: # Allows manual trigger from GitHub UI

 schedule:

 - cron: '0 0 * * *' # Optional: Runs daily at midnight UTC

 push:

 branches:

 - main

jobs:

 run-chunker:

 runs-on: ubuntu-latest

 steps:

 - name: Checkout repository

 uses: actions/checkout@v4

 - name: Set up Python

 uses: actions/setup-python@v5

 with:

 python-version: '3.12'

 - name: Install Dependencies

 run: |

 python -m pip install --upgrade pip

```
pip install pandas s3fs
```

- name: Configure AWS Credentials
uses: aws-actions/configure-aws-credentials@v4
with:
aws-access-key-id: \${{ secrets.AWS_ACCESS_KEY_ID }}
aws-secret-access-key: \${{ secrets.AWS_SECRET_ACCESS_KEY }}
aws-region: \${{ secrets.AWS_REGION }}
- name: Run S3 Chunker Script
run: |
python file_chunker_python.py

Step 5.3: Triggering the Workflow

- **Manual Run:** Go to the **Actions** tab in GitHub > Select **Run S3 File Chunker Pipeline** > Click **Run workflow**.
- **Automatic Run:** Pushing updates to the main branch or reaching the scheduled cron time will automatically trigger execution.